

PRÁCTICA 4:

CLOUD



<https://basmati.app/>

- Juan Manuel Valenzuela González
- Artur Vargas Carrión
- Jorge Repullo Serrano
- Eduardo González Bautista
- Rubén Oliva Zamora
- David Muñoz del Valle

- amcgil@uma.es
- arturvargas797@uma.es
- jorgers4@uma.es
- edugb@uma.es
- rubenoliva@uma.es
- davidmunvalle@uma.es

ÍNDICE DE CONTENIDO

1. TECNOLOGÍAS USADAS	1
1.1. BACKEND.....	1
1.1.1. <i>Lenguaje</i>	1
1.1.2. <i>Framework</i>	1
1.1.3. <i>Contenedores</i>	1
1.1.4. <i>Gestión de paquetes</i>	1
1.1.5. <i>Acceso a APIs externas</i>	1
1.1.6. <i>Base de datos</i>	1
1.2. FRONTEND.....	1
1.2.1. <i>Lenguaje</i>	1
1.2.2. <i>Framework</i>	1
1.2.3. <i>Build / bundler</i>	2
1.2.4. <i>Estilos</i>	2
1.2.5. <i>Cliente HTTP</i>	2
1.2.6. <i>Gestión de paquetes</i>	2
1.3. INTEGRACIONES EXTERNAS	2
1.3.1. <i>Almacenamiento</i>	2
1.3.2. <i>Calendarios</i>	2
1.3.3. <i>Geolocalización</i>	2
1.3.4. <i>Correo electrónico</i>	2
1.4. CLOUD.....	3
1.4.1. <i>Compute</i>	3
1.4.2. <i>Orquestación de contenedores</i>	3
1.4.3. <i>Reverse Proxy</i>	3
2. REQUISITOS CONSIDERADOS	4
2.1. CREACIÓN, EDICIÓN Y MANTENIMIENTO DE CALENDARIOS Y EVENTOS	4
2.2. IDENTIFICACIÓN DE CALENDARIOS: TÍTULO, ORGANIZADOR, PALABRAS CLAVE Y RELACIONES JERÁRQUICAS	4
2.3. IDENTIFICACIÓN DE EVENTOS: TÍTULO, HORA, DURACIÓN, LUGAR, ORGANIZADOR Y CONTENIDO MULTIMEDIA.....	4
2.4. BÚSQUEDA Y VISUALIZACIÓN DE CALENDARIOS	4
2.5. SISTEMA DE ALMACENAMIENTO NO RELACIONAL.....	5
2.6. ALMACENAMIENTO DE MEDIOS: IMÁGENES, ARCHIVOS Y MAPAS	5
2.7. GESTIÓN DE ACCESO CON OAUTH 2.0	5
2.8. OPTIMIZACIÓN DE SERVICIOS DE PAGO Y RATE LIMITS	5
2.9. COMENTARIOS Y NOTIFICACIONES	6
2.10. SERVICIOS WEB EXTERNOS Y DATOS ABIERTOS.....	6
2.11. ALOJAMIENTO EN CLOUD PÚBLICO.....	7
3. ARQUITECTURA DE LA APLICACIÓN.....	8
3.1. ARQUITECTURA DEL FRONTEND	8
3.2. DIAGRAMA DE COMPONENTES	9
3.2.1. <i>Cliente</i>	9
3.2.2. <i>Backend</i>	9
3.2.3. <i>Microservicios internos</i>	10
3.2.4. <i>APIs externas</i>	10
3.2.5. <i>Infraestructura de datos</i>	11
3.3. DIAGRAMA DE DESPLIEGUE	11
4. ESQUEMA DE NAVEGACIÓN.....	12

4.1.	FLUJO DE AUTENTICACIÓN.....	12
4.2.	FLUJO PRINCIPAL (CONTENIDO)	12
4.3.	CONFIGURACIÓN.....	12
4.4.	DIAGRAMA DE FLUJO DE NAVEGACIÓN.....	13
5.	BASE DE DATOS	14
5.1.	MANTENIMIENTO DEL DISEÑO DE LA BASE DE DATOS	14
5.2.	COLECCIONES DE LA BASE DE DATOS	14
5.2.1.	<i>Colección: users.....</i>	<i>14</i>
5.2.2.	<i>Colección: calendars.....</i>	<i>15</i>
5.2.3.	<i>Colección: events.....</i>	<i>15</i>
5.2.4.	<i>Colección: notifications.....</i>	<i>16</i>
5.2.5.	<i>Colección: geocode_cache.....</i>	<i>16</i>
5.3.	CREDENCIALES DE ACCESO	16
6.	DESPLIEGUE.....	17
6.1.	LOCAL.....	17
6.1.1.	<i>Arquitectura de contenedores</i>	<i>17</i>
6.1.2.	<i>Red interna</i>	<i>17</i>
6.1.3.	<i>Variables de entorno</i>	<i>17</i>
6.1.4.	<i>Ejecución.....</i>	<i>17</i>
6.2.	CLOUD.....	18
6.2.1.	<i>Análisis de alternativas y evolución del despliegue.....</i>	<i>18</i>
6.2.1.1.	<i>Estrategia de despliegue y seguridad.....</i>	<i>18</i>
6.2.2.	<i>Justificación técnica: AWS EC2 frente a AWS Lambda</i>	<i>18</i>
6.2.3.	<i>URL de despliegue.....</i>	<i>19</i>
7.	CAMBIOS EN LA API REST	20
7.1.	NUEVO SERVICIO: AUTH SERVICE (PUERTO 8005).....	20
7.1.1.	<i>Nuevos endpoints.....</i>	<i>20</i>
7.2.	CAMBIOS EN INTEGRATION SERVICE	21
7.2.1.	<i>Importación de calendarios.....</i>	<i>22</i>
7.2.1.1.	<i>Endpoints.....</i>	<i>22</i>
7.2.1.2.	<i>TeamUp.....</i>	<i>22</i>
7.2.1.3.	<i>Google Calendar.....</i>	<i>23</i>
	IMPORTANTE: restricción de acceso y validación de usuarios	23
7.2.2.	<i>Versionado V3 - Importaciones.....</i>	<i>23</i>
7.2.2.1.	<i>Arquitectura con patrón Abstract Factory.....</i>	<i>23</i>
7.2.2.2.	<i>Flujo de importación</i>	<i>23</i>
7.2.2.3.	<i>Separación de responsabilidades.....</i>	<i>24</i>
7.2.2.4.	<i>Extensibilidad</i>	<i>24</i>
8.	SERVICIOS EXTERNOS	25
8.1.	OPENSTREETMAP (MAPAS).....	25
8.1.1.	<i>Funcionalidades implementadas.....</i>	<i>25</i>
8.1.2.	<i>Componentes técnicos.....</i>	<i>25</i>
8.1.3.	<i>Consideraciones de integración</i>	<i>25</i>
8.2.	AWS S3 (IMÁGENES).....	25
8.2.1.	<i>Funcionalidades implementadas.....</i>	<i>25</i>
8.2.2.	<i>Componentes técnicos.....</i>	<i>26</i>
8.2.3.	<i>Configuración y límites:.....</i>	<i>26</i>
8.3.	SENDGRID (EMAIL).....	26
8.3.1.	<i>Funcionalidades implementadas.....</i>	<i>26</i>

8.3.2.	<i>Componentes técnicos</i>	26
8.3.3.	<i>Configuración de seguridad</i>	27
8.4.	GOOGLE OAUTH (AUTENTICACIÓN)	27
8.4.1.	<i>Funcionalidades implementadas</i>	27
8.4.2.	<i>Componentes técnicos</i>	27
8.5.	TEAMUP (IMPORTACIÓN DE CALENDARIOS)	28
8.5.1.	<i>Funcionalidades implementadas</i>	28
8.5.2.	<i>Componentes técnicos</i>	28
8.6.	GOOGLE CALENDAR (IMPORTACIÓN DE CALENDARIOS)	28
8.6.1.	<i>Funcionalidades implementadas</i>	28
8.6.2.	<i>Componentes técnicos</i>	28
8.6.3.	<i>Consideraciones comunes</i>	28
8.7.	AWS EC2 (DESPLIEGUE)	29
9.	NUEVA FUNCIONALIDAD	30
9.1.	MEJORA UI/UX DE LA PESTAÑA DE IMPORTACIÓN	30

1. Tecnologías usadas

1.1. Backend

1.1.1. Lenguaje

- **Python:** lenguaje principal del backend, utilizado para implementar la lógica de negocio y los servicios de la aplicación.

1.1.2. Framework

- **FastAPI:** framework web moderno para construir APIs REST rápidas y tipadas, con validación automática de datos, generación de documentación interactiva (OpenAPI/Swagger) y soporte nativo para programación asíncrona.

1.1.3. Contenedores

- **Docker:** cada servicio se ejecuta en su propio contenedor. Se define un Dockerfile por servicio para garantizar entornos reproducibles, facilitar el despliegue en distintos entornos y aislar dependencias. Orquestación usando docker-compose

1.1.4. Gestión de paquetes

- **pip:** gestor de paquetes de Python utilizado para instalar y versionar las dependencias del backend (frameworks, librerías auxiliares, clientes de APIs, etc.).

1.1.5. Acceso a APIs externas

- **httpx:** cliente HTTP asíncrono utilizado desde el backend para consumir servicios externos (por ejemplo, la API de Nominatim de OpenStreetMap), con soporte para timeouts y manejo de errores.

1.1.6. Base de datos

- **MongoDB Atlas:** base de datos NoSQL en la nube. Se utiliza para almacenar usuarios, calendarios, eventos, notificaciones y la caché de geocodificación, aprovechando esquemas flexibles y consultas sobre documentos anidados.

1.2. Frontend

1.2.1. Lenguaje

- **TypeScript:** lenguaje tipado para el frontend, que mejora la mantenibilidad del código, aporta autocompletado más rico y ayuda a detectar errores en tiempo de desarrollo.

1.2.2. Framework

- **React:** framework principal de UI para construir componentes reutilizables y vistas reactivas, gestionando el estado de forma declarativa y facilitando la composición de la interfaz.

1.2.3. Build / bundler

- **Vite:** herramienta de build y bundler moderno con recarga en caliente muy rápida durante el desarrollo y generación de bundles optimizados para producción.

1.2.4. Estilos

- **Tailwind CSS:** framework de utilidades CSS para diseño responsivo, que permite maquetar la interfaz mediante clases atómicas y mantener un diseño visual coherente.
- **PostCSS:** procesador de CSS en la cadena de build (autoprefijos, compatibilidad entre navegadores y optimización de estilos para producción).

1.2.5. Cliente HTTP

- **Axios:** cliente HTTP en el frontend para realizar peticiones REST al backend, con interceptores, configuración centralizada de cabeceras y manejo homogéneo de errores.

1.2.6. Gestión de paquetes

- **npm:** gestor clásico de paquetes de Node.js utilizado para instalar dependencias del frontend, ejecutar scripts y generar el `package-lock.json`.
- **pnpm:** gestor alternativo de paquetes, más rápido y eficiente en espacio gracias a su almacén compartido, útil cuando se trabaja con múltiples proyectos o monorepos.

1.3. Integraciones externas

1.3.1. Almacenamiento

- **Amazon S3:** almacenamiento de objetos para archivos estáticos e imágenes asociadas a eventos o calendarios, con control de acceso y posibilidad de servir el contenido a través de CDN.

1.3.2. Calendarios

- **Google Calendar:** importación y sincronización de calendarios desde cuentas de Google, para integrar agendas externas en la aplicación.
- **TeamUp Calendar:** importación de calendarios de TeamUp, permitiendo centralizar eventos procedentes de esta plataforma.

1.3.3. Geolocalización

- **OpenStreetMap (Nominatim API):** servicio de geocodificación y mapas. Se utiliza para convertir direcciones en coordenadas y viceversa, con un sistema de caché de geocodificación (módulo `geocode_cache.py`) almacenado en MongoDB para reducir latencia y llamadas repetidas.

1.3.4. Correo electrónico

- **SendGrid:** proveedor de email para el envío de correos transaccionales y notificaciones (por ejemplo, resúmenes, avisos de eventos o cambios en calendarios).

1.4. Cloud

1.4.1. Compute

- **AWS EC2:** servidor virtual en la nube utilizado para hospedar la aplicación. Se ejecuta una instancia Ubuntu 22.04 LTS que aloja todos los microservicios en contenedores Docker, permitiendo escalabilidad horizontal y gestión centralizada de la infraestructura.

1.4.2. Orquestación de contenedores

- **Docker Compose:** herramienta de orquestación utilizada para gestionar y coordinar todos los contenedores de la aplicación en el servidor EC2. Define la red interna, las dependencias entre servicios y facilita el despliegue y actualización de la arquitectura de microservicios.

1.4.3. Reverse Proxy

- **Nginx:** servidor web y reverse proxy configurado en el servidor EC2. Enruta las peticiones HTTP entrantes al frontend (puerto 5173) y al API Gateway (puerto 8000), proporciona compresión gzip, caché de archivos estáticos y configuración SSL para conexiones HTTPS.

2. Requisitos considerados

2.1. Creación, edición y mantenimiento de calendarios y eventos

Requisito: creación, edición y mantenimiento de calendarios de eventos y sus contenidos mediante el navegador. Los usuarios pueden crear calendarios sobre un tema y añadir, modificar o eliminar eventos y contenidos.

Implementación técnica:

- Arquitectura de **microservicios** con FastAPI (Python).
 - **calendar_service**: CRUD de calendarios.
 - **event_service**: CRUD de eventos.
- **Base de datos**: MongoDB (colecciones calendars y events).
- **Frontend**: React + TypeScript con arquitectura limpia.
- **APIs REST**: POST/PUT/DELETE /v1/calendars y /v1/events.

2.2. Identificación de calendarios: título, organizador, palabras clave y relaciones jerárquicas

Requisito: calendarios identificados por título, organizador, palabras clave, etc., con relaciones jerárquicas (ej.: calendario de curso engloba calendarios de asignaturas).

Implementación técnica:

- **Modelo con campos**: title, creator_external_id, creator_display_name, keywords, color.
- **Jerarquías**:
 - **parent_calendar_id**: referencia al padre.
 - **path**: array de IDs ancestros (Materialized Path).
- **Búsquedas**: por keywords, creador, texto completo, jerarquía.
- **Índices MongoDB**: keywords, path, parent_calendar_id, texto en title y description.

2.3. Identificación de eventos: título, hora, duración, lugar, organizador y contenido multimedia

Requisito: cada evento identifica título, hora de comienzo, duración, lugar, organizador, etc., con imágenes, archivos adjuntos, mapas, etc.

Implementación técnica:

- **Modelo con**: title, start_time, end_time, description, location, attachments, comments, recurrence.
- **Ubicaciones**: coordenadas (lat/lng) y soporte para Google Maps y OpenStreetMap.
- **Adjuntos**: almacenados en AWS S3 con compresión automática (JPEG 85%, max 1920x1080).
- **Recurrencia**: patrones diario, semanal, mensual, anual con excepciones.

2.4. Búsqueda y visualización de calendarios

Requisito: búsqueda por diversos criterios y visualización personalizada (completos o subcalendarios). Cada usuario ve conjuntamente los calendarios que ha decidido visualizar.

Implementación técnica:

- **Sistema de seguimiento**: campo followed_calendar_ids en usuarios.
- **Búsquedas**: texto completo, visibilidad, creador, keywords.
- **Visualización**: endpoint GET /v1/users/{id}/followed-calendars.
- **Frontend**: componente de búsqueda global con filtros.
- **Contador denormalizado**: subscriber_count en calendarios.

2.5. Sistema de almacenamiento no relacional

Requisito: uso de base de datos no relacional para persistencia.

Implementación técnica:

- **MongoDB** como base principal (base de datos basmati).
- **Colecciones:** users, calendars, events, notifications, geocode_cache.
- **Validación:** JSON Schema en MongoDB y Pydantic en aplicación.
- **Índices:** compuestos, texto completo, TTL para expiración automática.
- **Validadores JSON Schema** en cada colección para integridad.

2.6. Almacenamiento de medios: imágenes, archivos y mapas

Requisito: imágenes y archivos pueden alojarse en el sistema propio o en servicios externos (Cloudinary, Dropbox, Drive). Mapas configurables con OpenStreetMap, Google Maps u otros.

Implementación técnica:

- **Almacenamiento de imágenes:** AWS S3 con S3ImageService.
 - **Compresión automática** (JPEG 85%, redimensionado a 1920x1080).
 - **Endpoints:** /v2/integrations/s3/upload-direct y /upload (URLs presigned).
- **Geocodificación:** OpenStreetMap Nominatim API.
 - Geocodificación **directa e inversa**.
 - **Caché en MongoDB** (geocode_cache) para respetar límite de 1 req/seg.
- **Mapas:** soporte para Google Maps y OpenStreetMap.
 - Campo **map_provider** en eventos.
 - Componente **Location_Picker.tsx** en frontend con búsqueda y visualización.

2.7. Gestión de acceso con OAuth 2.0

Requisito: acceso identificado mediante OAuth 2.0 y cuentas externas (Google o Facebook). Solo el creador puede modificar su calendario y sus eventos.

Implementación técnica:

- **OAuth 2.0 con Google** en auth_service.
 - **Flujo:** /auth/google → /auth/google/callback → /auth/token.
 - **Scopes:** openid email profile y calendar.readonly.
- **Tokens JWT:** generación con jwt_handler.py, payload con external_id, email, display_name, provider.
- **Control de acceso:**
 - **Middleware de autenticación** en API Gateway.
 - **Verificación de creator_external_id** en operaciones de escritura.
 - Solo el **creador** puede **modificar** calendarios y eventos.
- **Integración:** creación automática de usuarios en primera autenticación.

2.8. Optimización de servicios de pago y rate limits

Requisito: optimización del acceso a servicios de pago o con cuotas/rates de uso.

Implementación técnica:

- **Caché** para APIs externas:
 - **Geocode cache** en MongoDB para OpenStreetMap (límite 1 req/seg).
 - **TTL automático** y contador de hits.
- Manejo de **rate limits**:
 - **OpenStreetMap:** User-Agent requerido, caché agresivo.
 - **SendGrid:** envío en batch, manejo de errores 429.

- **AWS S3:** compresión antes de subir, URLs presigned.
- **Optimización de costos:**
 - **Compresión automática** de imágenes (reduce almacenamiento S3).
 - **Caché de geocodificación** (reduce llamadas a APIs).
 - Agrupación de **notificaciones en resúmenes** diarios (reduce emails).
- **Manejo de errores:** detección de 429, retry con backoff exponencial, logging.

2.9. Comentarios y notificaciones

Requisito: comentarios en eventos. El creador recibe notificación cuando otro usuario comenta. Cada usuario configura si las notificaciones llegan por email o al conectarse.

Implementación técnica:

- **Sistema de comentarios:** modelo embebido en eventos (array de comentarios).
 - **Endpoint:** POST /v1/events/{event_id}/comments.
 - Cada **comentario** incluye: author_external_id, author_display_name, text, created_at.
- **Sistema de notificaciones:** microservicio notification_service.
 - **Tipos:** NEW_COMMENT, EVENT_UPDATE, CALENDAR_INVITE, EVENT_REMINDER.
- **Preferencias de usuario.**
- **Lógica de notificación (V2):**
 - Si **in_app=True**: crea notificación en notification_service.
 - Si **email=True** y **frequency="instant"**: envía email inmediato.
 - Si **email=True** y **frequency="daily"**: agrega a resumen diario.
- **Servicio de email:** SendGrid con EmailServiceV2.
 - **Templates HTML** para notificaciones de comentarios y resúmenes diarios.

2.10. Servicios web externos y datos abiertos

Requisito: uso de servicios web externos y/o conjuntos de datos abiertos relevantes (ej.: API de Teamup para importar calendarios). Política adecuada de consulta/actualización.

Implementación técnica:

- **Importación desde Google Calendar:**
 - **Servicio:** ImportService en integration_service.
 - **Endpoint:** POST /v2/integrations/import/google-calendar.
 - **Flujo:** autorización OAuth → obtención de calendarios → creación en Basmati → importación de eventos.
- **Importación desde Teamup:**
 - **Endpoint:** POST /v2/integrations/import/teamup.
 - **Flujo:** API Key → consulta API de Teamup → creación de calendario → importación de eventos (próximos 90 días).
 - **Manejo de errores HTTP** (401, 403, 404, 429).
- **Política de consulta:**
 - **Caché** de resultados de geocodificación.
 - **Respeto a límites** de APIs externas.
 - **Manejo** robusto de errores.
 - **Logging** de llamadas a APIs externas.
- **APIs externas utilizadas:**
 - **Google OAuth 2.0** (autenticación y Google Calendar).
 - **Teamup API** (importación de calendarios públicos).
 - **OpenStreetMap Nominatim** (geocodificación).

- **AWS S3** (almacenamiento de imágenes).
- **SendGrid** (envío de correos).

2.11. Alojamiento en cloud público

Requisito: alojamiento en cloud público (Amazon AWS, Vercel, etc.) para acceso web.

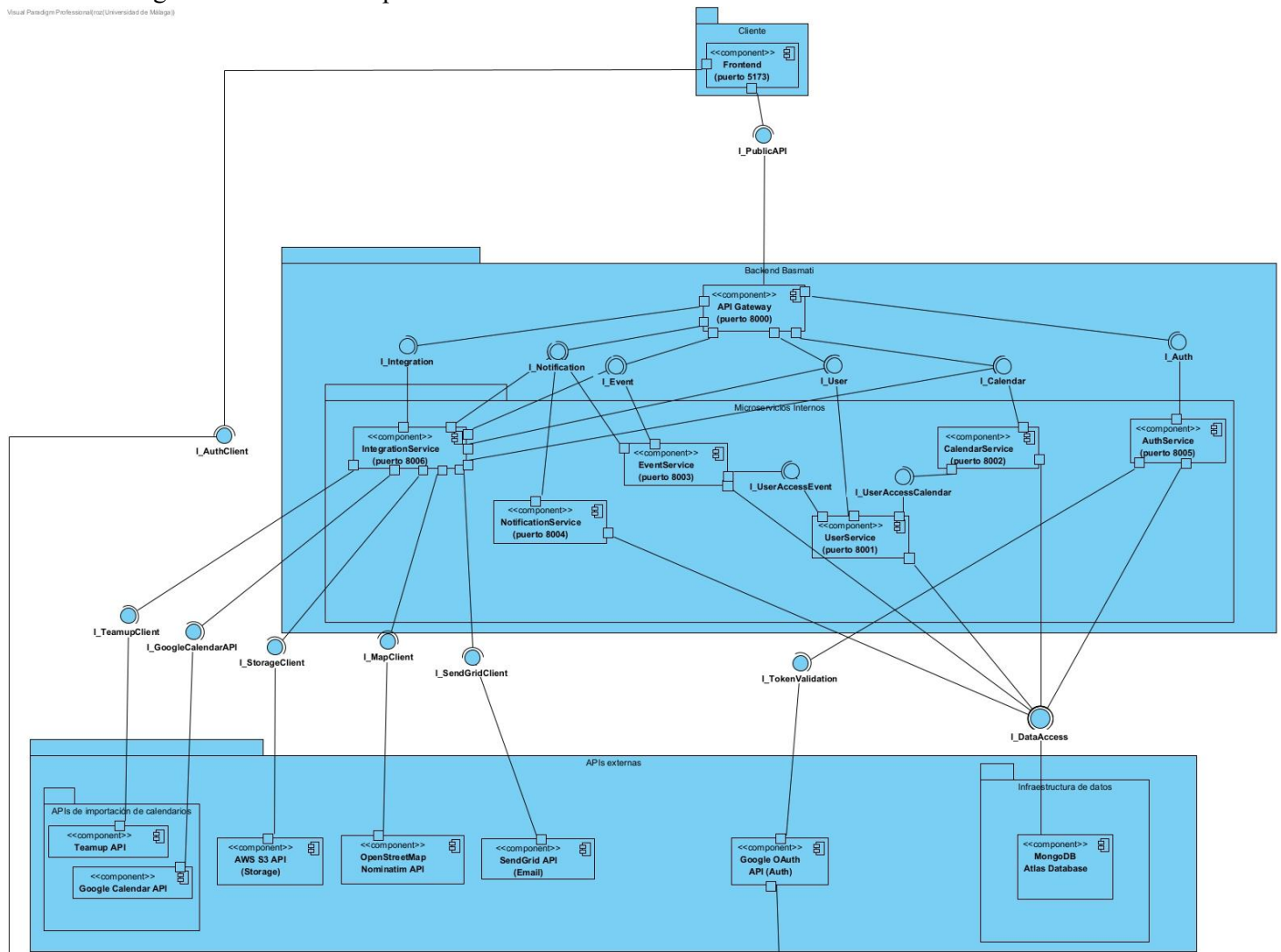
Implementación técnica:

- **Arquitectura de despliegue:**
 - **Plataforma:** Amazon AWS EC2 (Ubuntu 22.04 LTS).
 - **Orquestación:** Docker Compose para microservicios.
 - **Reverse Proxy:** Nginx para enrutamiento.
- **Scripts de despliegue:**
 - **deployment/setup-ec2.sh:** configuración inicial.
 - **deployment/deploy.sh:** despliegue automatizado con health checks, configuración de Nginx y rollback automático.
- **Configuración:**
 - Variables de entorno en **.env**.
 - **Secrets management** para credenciales.
 - **Health checks** para todos los servicios.
 - **MongoDB Atlas con whitelist** de IPs y backup automático.
 - **AWS S3 con CORS** y políticas de acceso.
 - **Monitoreo** con deployment/monitor.sh.

3.2. Diagrama de componentes

El siguiente diagrama de componentes representa la **arquitectura final implementada** para el sistema Basmati. Su objetivo es ofrecer una visión técnica completa de la estructura definitiva, detallando los puertos de comunicación, la especialización de cada microservicio y la integración con proveedores externos específicos.

La aplicación se ha consolidado como una arquitectura de **cliente-servidor basada en microservicios**. El cliente actúa como una interfaz reactiva que centraliza sus peticiones en un **API Gateway**, el cual distribuye la carga hacia un ecosistema de servicios internos independientes. Esta modularidad ha permitido un desarrollo paralelo eficiente y una escalabilidad granular de cada componente.



2 Diagrama de componentes de Basmati

A continuación, se describe con detalle cada uno de los elementos que componen el sistema:

3.2.1. Cliente

- **<<Componente UI>> Frontend (puerto 5173):** es el punto de entrada para el usuario final, desarrollado con **React** y **Vite**. Es responsable de la lógica de presentación, la gestión del estado en el lado del cliente y la comunicación asíncrona con el backend. Consume la API pública expuesta por el Gateway a través de la interfaz **I_PublicAPI**.

3.2.2. Backend

El backend agrupa la lógica de negocio y se organiza mediante un patrón de microservicios, donde cada componente está aislado en su propio contenedor o entorno de ejecución.

- **<<API Gateway>> (puerto 8000):** actúa como el **punto único de entrada** (single entry point) para todas las peticiones del Frontend. Sus responsabilidades finales son:
 - **Enrutamiento:** dirigir cada petición entrante al microservicio interno correspondiente según la ruta (puertos 8001 al 8006).
 - **Seguridad:** centralizar la validación de tokens y políticas de CORS antes de permitir el acceso a la red interna.
 - **Abstracción de red:** ofrece una capa de protección que oculta la topología interna y los puertos específicos de los microservicios.

3.2.3. Microservicios internos

Cada microservicio es un componente independiente con una responsabilidad única y su propia interfaz de comunicación.

- **UserService (puerto 8001):** responsable de la persistencia de perfiles de usuario y, fundamentalmente, de la gestión de permisos. Los demás servicios le consultan (`I_UserAccess`) para verificar si un usuario tiene autoridad para modificar un recurso.
- **CalendarService (puerto 8002):** se centra exclusivamente en la gestión de calendarios, sus propiedades, jerarquías y la organización estructural de los mismos.
- **EventService (puerto 8003):** contiene la lógica de negocio para la gestión de eventos (CRUD). Al igual que en el diseño inicial, este servicio asume la gestión de los **comentarios** por motivos estratégicos:
 - **Concepto de agregado (Aggregate):** un comentario no tiene sentido sin un evento; el evento es la raíz del agregado y gobierna su ciclo de vida.
 - **Consistencia de datos:** se garantiza la consistencia atómica al añadir comentarios y actualizar estados del evento en una misma operación, evitando transacciones distribuidas complejas.
 - **Delegación:** una vez creado un comentario, el servicio dispara una señal a través de la interfaz `I_Notification` para que el servicio correspondiente gestione el aviso al usuario.
- **NotificationService (puerto 8004):** orquesta el envío de notificaciones transaccionales. Recibe los disparadores del EventService y utiliza la API de **SendGrid** para el envío de correos electrónicos.
- **AuthService (puerto 8005):** microservicio especializado en la lógica de autenticación. Gestiona el flujo de login y la validación de identidad interactuando con proveedores externos (Google OAuth).
- **IntegrationService (puerto 8006):** gestiona la comunicación con el ecosistema externo. Se encarga de la importación de datos desde **Teamup** y **Google Calendar**, pero también centraliza el acceso a servicios de infraestructura como el almacenamiento en **AWS S3** y la geolocalización mediante **OpenStreetMap**.

3.2.4. APIs externas

Servicios de terceros que dotan a Basmati de capacidades avanzadas de forma segura y eficiente:

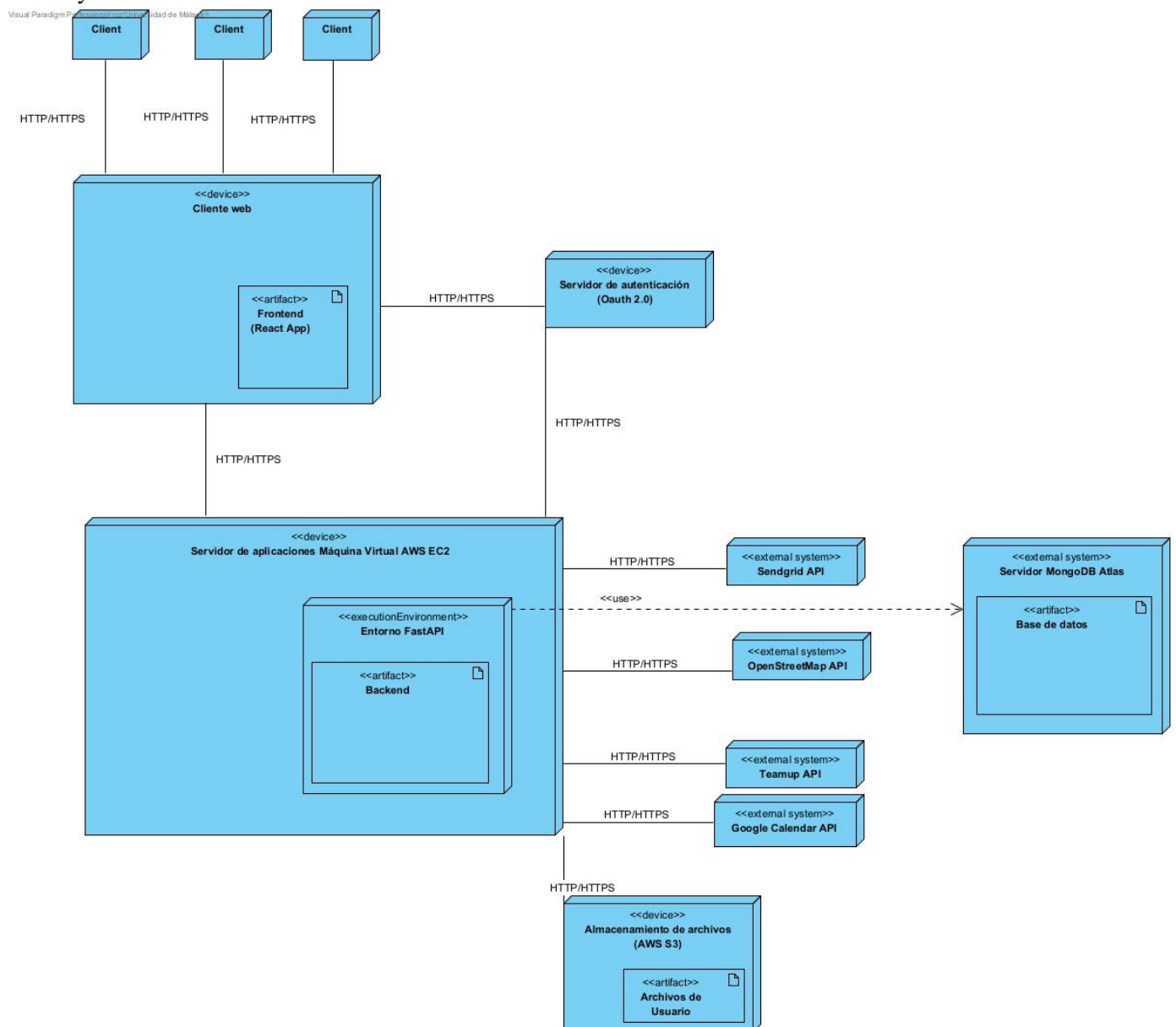
- **Google OAuth API (`I_TokenValidation`):** gestiona la validación de identidad mediante el protocolo OAuth 2.0, permitiendo un acceso seguro basado en tokens sin almacenar contraseñas críticas en el sistema.
- **Google Calendar API (`I_GoogleCalendarAPI`):** permite la sincronización bidireccional de eventos con la plataforma de Google.
- **Teamup API (`I_TeamupClient`):** interfaz para la importación de calendarios específicos de la plataforma Teamup mediante peticiones HTTP.
- **AWS S3 API (`I_StorageClient`):** provee el almacenamiento persistente para archivos pesados, imágenes y documentos adjuntos a los eventos, garantizando alta disponibilidad.
- **OpenStreetMap Nominatim API (`I_MapClient`):** utilizada para la geocodificación de direcciones y la visualización de mapas, ofreciendo una alternativa abierta y eficiente.
- **SendGrid API (`I_SendGridClient`):** servicio especializado para asegurar la entrega de notificaciones por email, evitando que los correos sean marcados como spam.

3.2.5. Infraestructura de datos

- **PersistenceService (MongoDB Atlas Database):** este componente actúa como la capa de acceso a datos (DAL) final. En la implementación terminada, se utiliza **MongoDB Atlas** (DBaaS) para garantizar la persistencia en la nube.
- **I_DataAccess:** todos los microservicios acceden a la base de datos exclusivamente a través de esta interfaz común.
 - **Desacoplamiento:** los servicios no conocen la implementación física de MongoDB; solo interactúan con el contrato definido en la interfaz.
 - **Intercambiabilidad:** al estar centralizada en el PersistenceService, si se decidiera cambiar a otra base de datos en el futuro, solo se modificaría este componente, dejando intacta la lógica de los microservicios.
 - **Mantenimiento:** facilita la optimización de consultas y la gestión de conexiones desde un único punto de control.

3.3. Diagrama de despliegue

Para complementar la visión lógica del sistema, el siguiente diagrama de despliegue detalla la infraestructura física y el entorno de ejecución de Basmati. En él se ilustra cómo los 8 microservicios contenedorizados mediante Docker se distribuyen sobre la instancia de **AWS EC2** en el entorno cloud.



3 Diagrama de despliegue de Basmati

4. Esquema de navegación

El esquema de navegación de la aplicación se organiza en tres flujos principales: autenticación, gestión de contenido y configuración.

En la página siguiente se muestra cómo quedaría finalmente la navegación de la versión final de Basmati.

4.1. Flujo de autenticación

Ruta	Página	Descripción
/login	Login_Page	Página de inicio de sesión con Google OAuth
/auth/callback	Auth_Callback_Page	Procesa el retorno de Google y redirige al dashboard
/	-	Redirige automáticamente a /login

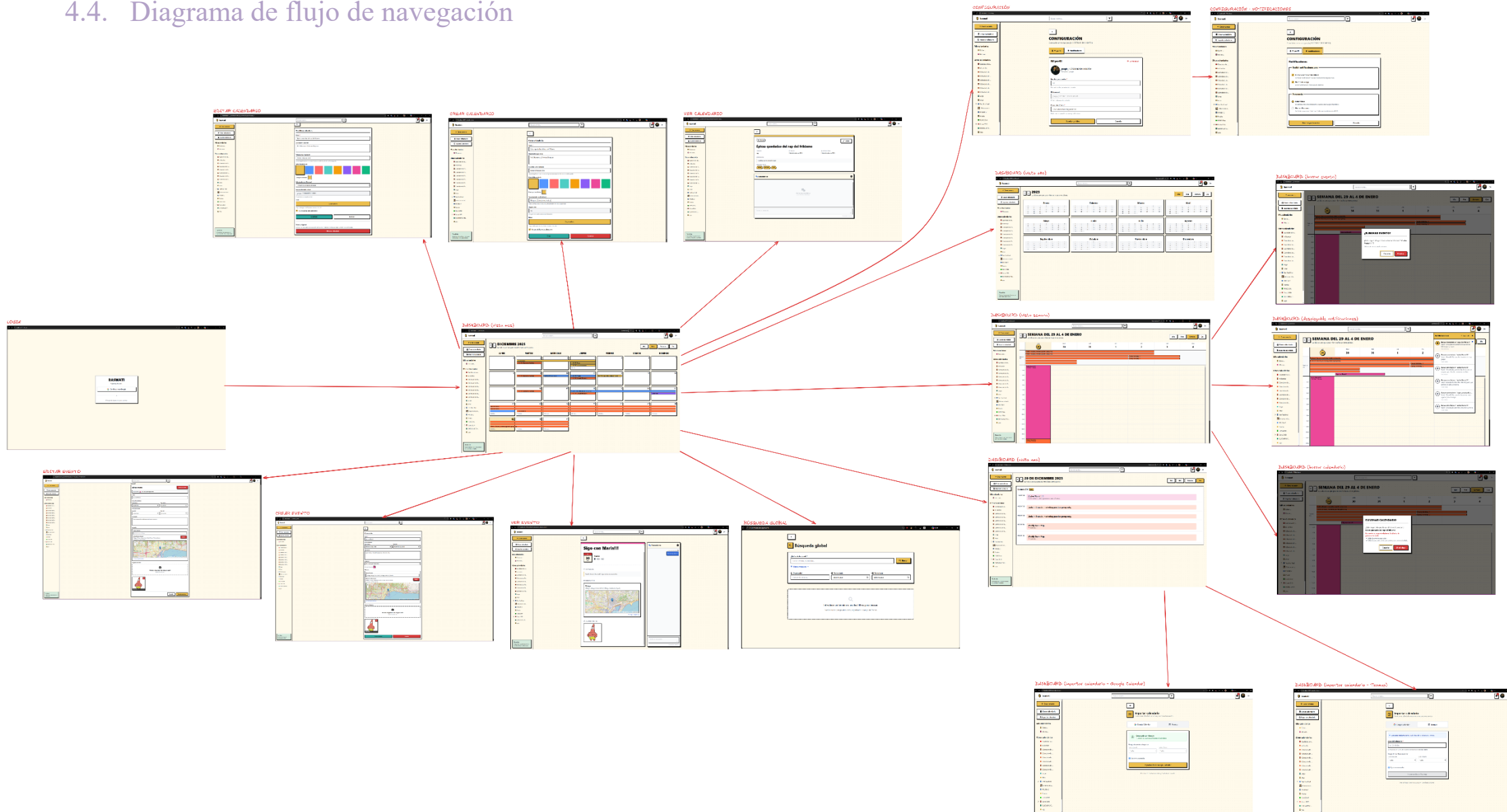
4.2. Flujo principal (contenido)

Ruta	Página	Descripción
/dashboard	Dashboard_Page	Vista principal con calendarios y eventos del usuario
/search	Search_Page	Búsqueda de calendarios y eventos
/calendars/new	Create_Calendar_Page	Formulario de creación de calendario
/calendars/import	Import_Calendar_Page	Importación desde Google Calendar o Teamup
/calendars/:id	Calendar_Detail_Page	Detalle de un calendario con sus eventos
/calendars/edit/:id	Edit_Calendar_Page	Edición de un calendario existente
/events/new	Create_Event_Page	Formulario de creación de evento
/events/:id	Event_Detail_Page	Detalle de un evento con mapa y adjuntos
/events/edit/:id	Edit_Event_Page	Edición de un evento existente

4.3. Configuración

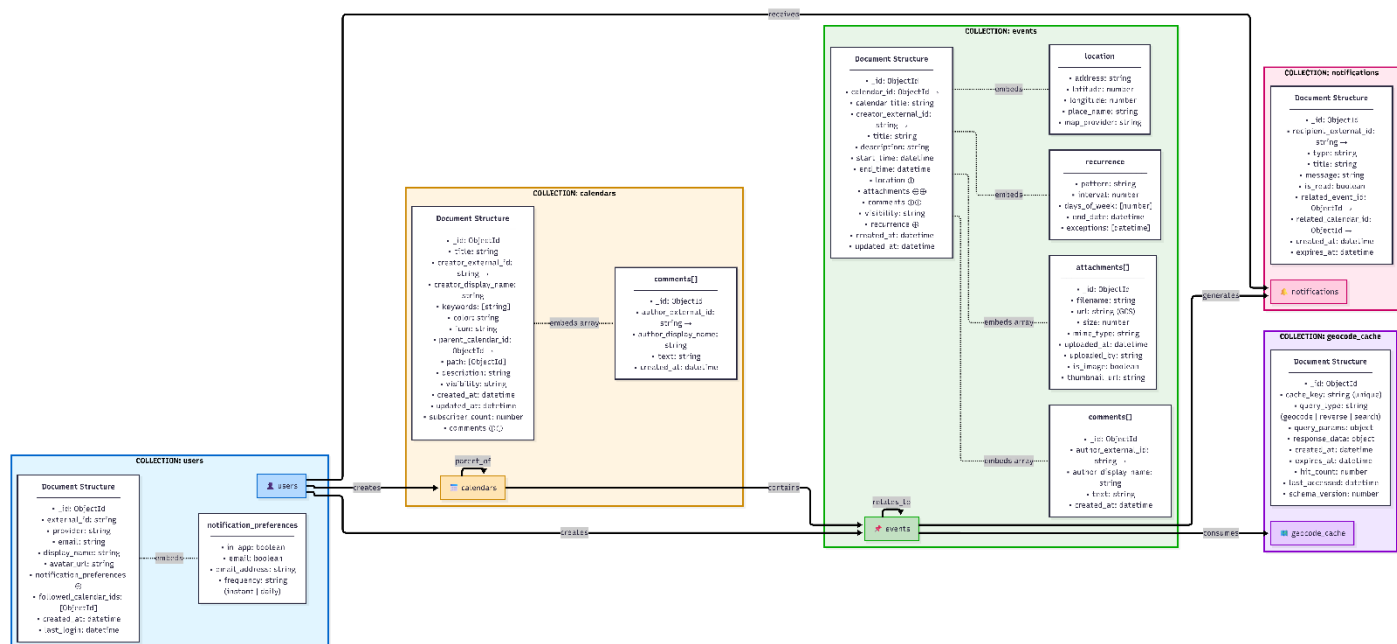
Ruta	Página	Descripción
/settings	Settings_Page	Preferencias del usuario (notificaciones, perfil)

4.4. Diagrama de flujo de navegación



4 Esquema de navegación de Basmati

5. Base de datos



5 Diagrama de la base de datos NoSQL

5.1. Mantenimiento del diseño de la base de datos

El **robusto diseño previo de la base de datos** ha permitido no tener que hacer cambios en la base de datos con respecto a la práctica anterior. La base de datos se mantiene sin alteraciones, así como la combinación de **desnormalización** (datos duplicados para lecturas eficientes) y **referenciación mediante identificadores** para expresar relaciones entre entidades.

5.2. Colecciones de la base de datos

5.2.1. Colección: users

Esta colección almacena la información de los perfiles de usuario y sus configuraciones personales.

- **_id**: identificador único interno del documento.
- **external_id**: ID único proveniente de OAuth.
- **provider**: el servicio de autenticación utilizado.
- **Email**: email del usuario
- **display_name**: nombre del usuario
- **avatar_url**: url de la imagen de perfil del usuario.
- **notification_preferences**: objeto que define si el usuario recibe avisos por app o email, y con qué frecuencia.
 - **in_app**: booleano que indica si quiere las notificaciones dentro de la app.
 - **email**: booleano que indica si quiere las notificaciones por email.
 - **email_address**: dirección a la que llegan las notificaciones.
 - **frequency**: frecuencia con la que se mandan notificaciones.
- **followed_calendar_ids**: lista de IDs de calendarios a los que el usuario está suscrito.
- **created_at**: fecha de creación del usuario.
- **last_login**: fecha del último acceso del usuario.

5.2.2. Colección: calendars

Representa los contenedores de eventos. Pueden ser personales o compartidos.

- **_id**: identificador único del calendario.
- **title**: nombre del calendario.
- **description**: detalle del calendario.
- **creator_external_id**: external id del creador del calendario.
- **creator_display_name**: nombre del creador del calendario.
- **keywords**: etiquetas para búsqueda y categorización.
- **color**: color del calendario en la interfaz.
- **icon**: icono del calendario en la interfaz.
- **parent_calendar_id**: id del calendario padre.
- **path**: camino para la jerarquía (sub-calendarios).
- **visibility**: define si es público o privado.
- **subscriber_count**: contador de cuántas personas siguen este calendario.
- **comments (Array embebido)**: comentarios dejados por los usuarios en el calendario.
 - **_id**: id del comentario.
 - **author_external_id**: id del autor del comentario.
 - **author_display_name**: nombre del autor del comentario.
 - **text**: texto del comentario.
 - **created_at**: fecha de creación del comentario.

5.2.3. Colección: events

Representa las citas o actividades específicas dentro de un calendario.

- **_id**: identificador único del evento.
- **calendar_id**: referencia al ID del calendario al que pertenece.
- **calendar_title**: nombre del calendario (desnormalizado para acceso rápido).
- **creator_external_id**: ID externo del usuario que creó el evento.
- **title**: título del evento.
- **description**: detalle o cuerpo del evento.
- **start_time**: fecha y hora de inicio.
- **end_time**: fecha y hora de finalización.
- **visibility**: define si el evento es público o privado.
- **created_at**: fecha de creación
- **updated_at**: fecha de última modificación.
- **location**: información geográfica del evento.
 - **address**: dirección física.
 - **latitude**: latitud del lugar del evento.
 - **longitude**: longitud del lugar del evento.
 - **place_name**: nombre del lugar.
 - **map_provider**: proveedor del mapa.
- **recurrence**: reglas de repetición del evento.
 - **pattern**: patrón de repetición (diario, semanal, etc.).
 - **interval**: frecuencia del intervalo.
 - **days_of_week**: días de la semana en que ocurre.
 - **end_date**: fecha en la que termina la recurrencia.
 - **exceptions**: lista de fechas específicas donde no aplica la recurrencia.
- **attachments (Array embebido)**: archivos adjuntos al evento.
 - **_id**: id del archivo.
 - **filename**: nombre del archivo.
 - **url**: enlace al almacenamiento (GCS).

- **mime_type**: tipo de archivo.
- **uploaded_at**: fecha de subida del archivo.
- **uploaded_by**: id del usuario que sube el archivo
- **is_image**: booleano que indica si es una imagen.
- **thumbnail_url**: url de la miniatura.
- **comments** (Array embebido): comentarios específicos del evento.
 - **_id**: id del comentario.
 - **author_external_id**: id del autor del comentario.
 - **autor_display_name**: nombre del autor del comentario.
 - **text**: texto del comentario.
 - **created_at**: fecha de creación del comentario.

5.2.4. Colección: notifications

Registra las alertas y avisos generados para los usuarios.

- **_id**: identificador único de la notificación.
- **recipient_external_id**: ID del usuario que debe recibir la notificación.
- **type**: tipo de notificación.
- **title**: título breve del aviso.
- **message**: contenido detallado de la notificación.
- **is_read**: estado de lectura (verdadero/falso).
- **related_event_id**: ID del evento relacionado (si aplica).
- **related_calendar_id**: ID del calendario relacionado (si aplica).
- **created_at**: fecha de generación.
- **expires_at**: fecha en la que la notificación dejará de ser válida o se borrará.

5.2.5. Colección: geocode_cache

Almacena resultados de peticiones de mapas para optimizar el rendimiento y costos.

- **_id**: identificador único del registro de caché.
- **cache_key**: llave única de búsqueda.
- **query_type**: tipo de consulta realizada.
- **query_params**: objeto con los parámetros originales de la búsqueda.
- **response_data**: objeto con la respuesta completa recibida del proveedor de mapas.
- **created_at**: fecha creación del caché.
- **expires_at**: fecha de expiración del caché.
- **hit_count**: contador de cuántas veces se ha servido este dato desde la caché.
- **last_accessed**: última vez que se consultó este registro.
- **schema_version**: versión del esquema de datos para migraciones futuras.

5.3. Credenciales de acceso

Nuestra base de datos está desplegada utilizando **MongoDB Atlas**. Esto significa que no está alojada en un servidor local (como localhost), sino **en la nube**. La URI del despliegue de la base de datos es la siguiente:

```
mongodb+srv://<db_username>:<db_password>@basmaticcluster.revmyok.mongodb.net/basmati?retryWrites=true&w=majority
```

Esta cadena no contiene la contraseña real ni el usuario por cuestiones de seguridad. La URI completa se configura mediante la variable de entorno **MONGO_URI** (en el fichero .env de la aplicación).

6. Despliegue

6.1. Local

Para el despliegue en entorno de desarrollo local, se utiliza **Docker** y **Docker Compose** como herramientas de contenerización. Esta decisión permite replicar el entorno de producción de forma consistente, aislar los servicios y simplificar la configuración de dependencias.

6.1.1. Arquitectura de contenedores

La aplicación se compone de **8 contenedores** orquestados mediante Docker Compose:

Servicio	Puerto	Descripción
frontend	5173	Aplicación React con Vite
api-gateway	8000	Punto de entrada centralizado para el backend
user-service	8001	Gestión de usuarios y autenticación
calendar-service	8002	Gestión de calendarios
event-service	8003	Gestión de eventos
notification-service	8004	Gestión de notificaciones
auth-service	8005	Gestión de autenticación con Google OAuth
integration-service	8006	Gestión de integración de servicios externos (imágenes, mapas, importaciones...)

6.1.2. Red interna

Todos los contenedores se conectan a una red Docker común (`basmati-network`) que permite la comunicación entre servicios mediante nombres de host internos (ej: `http://user-service:8001`).

6.1.3. Variables de entorno

La configuración sensible (credenciales de MongoDB, claves OAuth, API keys) se gestiona mediante un archivo `.env` en la raíz del proyecto, que Docker Compose inyecta automáticamente en cada contenedor.

6.1.4. Ejecución

Para iniciar el entorno completo:

```
docker-compose up -build
```

Una vez levantados los servicios, la aplicación está disponible en:

- **Frontend:** `http://localhost:5173`
- **API Gateway:** `http://localhost:8000`
- **Documentación OpenAPI:** `http://localhost:8000/docs`

6.2. Cloud

Para el despliegue de nuestra aplicación, optamos por un enfoque de infraestructura tradicional basado en IaaS (Infrastructure as a Service) mediante el uso de **AWS EC2**. Esta decisión nos ha permitido tener un control absoluto sobre el entorno de ejecución, gestionando directamente una instancia.

Llegar a la configuración actual no fue una decisión de un solo día. Fue un proceso de ensayo y error donde probamos diferentes alternativas antes de darnos cuenta de que necesitábamos el control total que nos ofrece una infraestructura tradicional.

6.2.1. Análisis de alternativas y evolución del despliegue

Nuestra primera opción sobre el despliegue fue una opción *serverless*, ejecutar el backend sobre AWS Lambda, un servicio de computación que permite ejecutar código sin gestionar servidores y el frontend sobre el alojamiento estático S3, el cual también es usado para el almacenamiento de imágenes.

Para organizar y desplegar el backend en AWS Lambda, utilizábamos las siguientes herramientas:

- **AWS SAM (Serverless Application Model):** es un marco de trabajo (framework) de código abierto que simplifica la construcción de aplicaciones serverless. Nos permite definir los recursos (como las funciones Lambda y las bases de datos) mediante archivos de configuración simples, automatizando el despliegue.
- **Magnum:** es un adaptador que actúa como puente. Permite que aplicaciones web modernas (como las desarrolladas en FastAPI o Flask) funcionen dentro de AWS Lambda, traduciendo las peticiones del API Gateway al formato que entiende la aplicación.

6.2.1.1. Estrategia de despliegue y seguridad

Durante el desarrollo, evaluamos el uso de contenedores (Docker), pero decidimos descartarlos por los siguientes motivos:

- **Limitaciones de la capa gratuita:** el almacenamiento de imágenes de contenedor privadas en AWS (mediante ECR) puede exceder los límites gratuitos si las imágenes son muy voluminosas.
- **Riesgos de seguridad:** para evitar costes, una alternativa sería usar repositorios públicos, pero esto expondría el código y la lógica del negocio. Además, la gestión de secretos en contenedores mal configurados puede ser vulnerable; si un contenedor es público, cualquier dato sensible embebido sería fácilmente accesible.

Para la **gestión de secretos**, aunque consideramos AWS Secrets Manager, decidimos buscar alternativas más económicas, ya que este servicio tiene un coste fijo por secreto almacenado que no se ajustaba a nuestro presupuesto de "coste cero" o mínimo.

Para la **interfaz de usuario**, nuestra primera opción fue utilizar Amazon S3 (Simple Storage Service). En lugar de alquilar un servidor para servir la web, subimos los archivos estáticos (HTML, CSS, JS) directamente a un "bucket" de S3 configurado para hosting web.

- **Bajo coste:** el almacenamiento y la transferencia de datos en S3 son extremadamente económicos.
- **Alta disponibilidad:** AWS se encarga de que el contenido esté siempre disponible sin necesidad de gestionar servidores web como Nginx o Apache.

6.2.2. Justificación técnica: AWS EC2 frente a AWS Lambda

Tras un análisis exhaustivo de las alternativas de computación en la nube, evaluamos la posibilidad de utilizar AWS Lambda. Sin embargo, descartamos esta opción debido a que su arquitectura serverless exige que el código sea nativo

y esté fragmentado en funciones independientes. Migrar nuestra aplicación multiservicios a este modelo habría supuesto una reestructuración compleja y costosa del código fuente.

En contraste, AWS EC2 nos ofrece una ventaja competitiva en términos de profesionalidad y eficiencia operativa. Al operar sobre una "filosofía tradicional", hemos logrado un despliegue más natural, permitiendo que la aplicación mantenga su estructura original, sin embargo, este sistema no es perfecto y nos dejamos atrás todos los beneficios que nos hicieron plantearnos la arquitectura serverless en Basmati.

Característica	Solución supuesta (Lambda + S3)	Solución real (EC2 / ECS)
Modelo de coste	Pago por uso estricto. Si no hay tráfico, el gasto es prácticamente cero.	Pago por tiempo/recursos. Se factura por horas de encendido, se use o no (Gratis primer mes)
Escalabilidad	Automática y nativa. Soporta incrementos de tráfico instantáneos sin configuración manual.	Manual. Requiere configurar grupos de autoescalado y balanceadores de carga.
Mantenimiento	AWS gestiona el sistema operativo	El desarrollador debe gestionar el servidor.
Disponibilidad del Frontend	Alta disponibilidad nativa. S3 sirve el contenido desde la red de AWS sin caídas de servidor.	Dependiente del servidor. Si la instancia EC2 falla o se satura, el sitio deja de estar disponible.

Nos decantamos definitivamente por **AWS EC2** no solo por su fácil implementación, sino por la mantenibilidad y escalabilidad que ofrece sobre la estructura en la que se basa nuestra aplicación. Esta decisión nos permite centrar nuestros esfuerzos en la evolución de la aplicación y no en la resolución de las limitaciones impuestas por la infraestructura de AWS Lambda.

6.2.3. URL de despliegue

Nuestra aplicación, Basmati, se encuentra desplegada y accesible de forma pública a través de la siguiente URL:

<https://basmati.app/>

Con el objetivo de garantizar la integridad y la privacidad de los datos de nuestros usuarios, hemos implementado un robusto protocolo de seguridad mediante:

- **Cifrado SSL/TLS:** el tráfico entre el cliente y el servidor está totalmente cifrado bajo el protocolo HTTPS.
- **Certificación Let's Encrypt:** hemos utilizado la autoridad certificadora Let's Encrypt, un estándar de la industria que garantiza la autenticidad de nuestro sitio.
- **Gestión con Certbot:** para la implementación y renovación automática de estos certificados, hemos configurado Certbot dentro de nuestra instancia de Ubuntu. Esto nos asegura que la conexión sea siempre segura sin interrupciones por caducidad de claves.

7. Cambios en la API REST

La evolución de la API en esta iteración se ha centrado en la implementación de la autenticación y la finalización de los servicios de integración que quedaron pendientes o incompletos en la fase anterior.

7.1. Nuevo servicio: Auth Service (puerto 8005)

Se ha desarrollado e integrado el servicio de autenticación completo, requisito indispensable que impedía el avance de otras funcionalidades en la entrega pasada.

- **Implementación de Google OAuth 2.0:** el sistema ya no depende de simulaciones. Se validan las credenciales reales del usuario contra los servidores de Google, utilizando el flujo de autorización estándar con códigos temporales.
- **Gestión de sesión:** generación de tokens JWT que permiten identificar de forma segura al usuario en el backend, desbloqueando así la capacidad de vincular datos externos (calendarios) a cuentas de usuarios específicas.
- **Flujo completo de autenticación**
 - `/v1/auth/google` para iniciar el flujo,
 - `/v1/auth/google/callback` para recibir la respuesta de Google, y
 - `/v1/auth/token` valida el código temporal recibido de Google y devuelve el token JWT del usuario autenticado. Este token debe utilizarse posteriormente en la cabecera `Authorization` del resto de operaciones de la API.

7.1.1. Nuevos endpoints

GET `/v1/auth/google`

Retorna la URL para iniciar el flujo OAuth con Google.

Parámetro	Tipo	Requerido	Descripción
<code>redirect_to</code>	string (query)	-	URL a la que redirigir después del login (ej. <code>/dashboard</code>).

Ejemplo de uso: GET `/v1/auth/google`

GET `/v1/auth/google/callback`

Endpoint al que Google redirige después de la autorización. Genera un código temporal para obtener el token.

Parámetro	Tipo	Requerido	Descripción
<code>code</code>	string (query)	Sí	Código de autorización de Google.
<code>state</code>	string (query)	-	URL de redirección original. Default value : <code>/dashboard</code>

Ejemplo de uso: GET `/v1/auth/google/callback?code=4/0Afge...&state=xyz123`

POST `/v1/auth/google/verify`

Verifica un ID token de Google y retorna un token de sesión.

Parámetro	Tipo	Requerido	Descripción
id_token	string	Sí	El token JWT proporcionado por Google tras el login en el cliente.

Ejemplo de uso: POST /v1/auth/google/verify

POST /v1/auth/google/verify

Verifica si un token JWT de sesión es válido.

Parámetro	Tipo	Requerido	Descripción
token	string	Sí	Token de sesión o acceso de la aplicación a verificar.

Ejemplo de uso: POST /v1/auth/verify

GET /v1/auth/me

Retorna los datos del usuario basado en el token de autorización.

Parámetro	Tipo	Requerido	Descripción
authorization	string (query)	-	Bearer token (alternativa al header).

Ejemplo de uso: GET /v1/auth/me

GET /v1/auth/token

Devuelve el token JWT del usuario autenticado para usar en la API.

Parámetro	Tipo	Requerido	Descripción
code	string (query)	-	Código de autorización temporal recibido del callback.

Ejemplo de uso: GET /v1/auth/token

POST /v1/auth/logout

Endpoint informativo de logout. El logout real se realiza en el frontend eliminando el token.

Parámetro	Tipo	Requerido	Descripción
-	-	-	-

Ejemplo de uso: POST /v1/auth/logout

7.2. Cambios en Integration Service

Este servicio ha sufrido una refactorización profunda para solucionar los errores de diseño detectados anteriormente y habilitar las conexiones externas reales.

7.2.1. Importación de calendarios

7.2.1.1. Endpoints

GET /v3/integrations/imports/providers

Retorna la lista de proveedores de calendario soportados con sus capacidades.

Parámetro	Tipo	Requerido	Descripción
-	-	-	-

Ejemplo de uso: GET /v3/integrations/imports/providers

GET /v3/integrations/imports/providers/{provider}

Retorna información detallada de un proveedor específico.

Parámetro	Tipo	Requerido	Descripción
provider	string (path)	Sí	El identificador del proveedor específico.

Ejemplo de uso: GET /v3/integrations/imports/providers/Google

POST /v3/integrations/imports/

Endpoint genérico para importar desde cualquier proveedor soportado.

Permite especificar el proveedor dinámicamente en el request.

Parámetro	Tipo	Requerido	Descripción
-	-	-	-

Ejemplo de uso: GET /v3/integrations/imports/

7.2.1.2. TeamUp

Se ha finalizado la implementación de este módulo, corrigiendo los **fallos graves** reportados en la entrega anterior:

- **Vinculación de usuario:** los calendarios y eventos importados ahora se asocian correctamente al `user_external_id` del usuario autenticado
- **Gestión de credenciales (API Keys):** se han sustituido las claves incorrectas de la versión anterior. El sistema ahora implementa correctamente la arquitectura de **API Key de desarrollador** (para la autorización) combinada con el **Calendar ID** proporcionado por el usuario, permitiendo la lectura efectiva de eventos.
- **Finalización del servicio:** la lógica de importación, que estaba incompleta, es ahora funcional, permitiendo mapear los eventos de TeamUp al modelo interno de la aplicación.

POST /v3/integrations/imports/teamup

Importa calendarios desde Teamup usando el patrón Abstract Factory.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	Sí	JSON con <code>user_external_id</code> y el array de <code>calendar_ids</code> .

Ejemplo de uso: GET /v3/integrations/imports/teamup

7.2.1.3. Google Calendar

Habiendo superado la limitación técnica de la falta de OAuth, se ha procedido al desarrollo de esta integración:

- **Acceso a la API:** gracias al nuevo *Auth Service*, el sistema ahora captura y utiliza el `access_token` del usuario.
- **Lectura de eventos:** se ha implementado el consumo del endpoint `primary` de la API de Google Calendar utilizando el scope `calendar.readonly`, permitiendo importar la agenda personal del usuario, funcionalidad que fue prorrogada en la práctica anterior y que ahora está operativa.
- **Paginación automática:** se implementó el manejo de paginación para calendarios con gran cantidad de eventos, procesando hasta 250 eventos por página.

POST /v3/integrations/imports/google

Importa calendarios desde Google Calendar usando el patrón Abstract Factory.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	Sí	JSON con <code>user_external_id</code> , <code>access_token</code> y el array de <code>calendar_ids</code> .

Ejemplo de uso: GET /v3/integrations/imports/google

IMPORTANTE: restricción de acceso y validación de usuarios

La implementación actual de la importación de calendarios de Google se realiza mediante una integración directa con la API a través del flujo de **Google OAuth 2.0**, descartando el uso de archivos manuales en formato `.ics` para ofrecer una sincronización real.

Debido a que la aplicación se encuentra en entorno de desarrollo y no ha sido sometida al proceso de verificación oficial de Google (el cual puede extenderse por más de 3 meses), el acceso está limitado por políticas de seguridad de Google Cloud. Por tanto, para probar el inicio de sesión o la importación, es **estrictamente necesario** que el correo electrónico del usuario esté registrado previamente en la lista de "**Usuarios de prueba**" en la Google Cloud Console. Sin esta configuración previa, el sistema denegará el acceso al token de autenticación por motivos de seguridad.

7.2.2. Versionado V3 - Importaciones

La versión V3 de importaciones se implementa con el patrón Abstract Factory para separar responsabilidades y facilitar la extensión a nuevos proveedores.

7.2.2.1. Arquitectura con patrón Abstract Factory

Se definen tres productos abstractos:

- **ICalendarConnector:** comunicación HTTP con la API externa (autenticación, requests, manejo de errores de red).
- **IEventParser:** transformación de datos crudos del proveedor a objetos de dominio (normalización de fechas, mapeo de campos, validación).
- **ICalendarImporter:** orquestación de la importación completa (coordinación entre Connector y Parser, creación de calendarios y eventos en Basmati).

Cada proveedor (Google, Teamup) implementa una factoría concreta (`GoogleImportFactory`, `TeamupImportFactory`) que crea la familia de componentes compatibles entre sí.

7.2.2.2. Flujo de importación

El servicio `ImportServiceV3` instancia la factoría según el proveedor. La factoría crea el `Importer` con sus dependencias (`Connector` y `Parser`).

El Importer ejecuta:

1. Obtiene información del calendario externo mediante el `Connector`.
2. Parsea la información con el `Parser`.
3. Crea el calendario en Basmati mediante el `CalendarService`.
4. Obtiene eventos del calendario externo con el `Connector` (con paginación automática si es necesario).
5. Parsea los eventos con el `Parser` a objetos `ExternalEvent`.
6. Crea cada evento en Basmati mediante el `EventService`.

7.2.2.3. Separación de responsabilidades

- **Connector:** solo comunicación HTTP; no conoce la estructura interna de Basmati.
- **Parser:** solo transformación de datos; no realiza llamadas HTTP.
- **Importer:** orquesta el proceso; usa `Connector` y `Parser`, pero no implementa la lógica específica de cada proveedor.

7.2.2.4. Extensibilidad

Para añadir un nuevo proveedor (por ejemplo, Outlook), se implementan las tres interfaces en una carpeta dedicada y se crea su factoría concreta, registrándola en el servicio. No se modifica código existente.

8. Servicios externos

8.1. OpenStreetMap (mapas)

El proyecto utiliza OpenStreetMap como proveedor principal de servicios geográficos. La integración se divide en dos áreas: servicios de geocodificación (vía API Nominatim) y visualización de mapas interactivos.

8.1.1. Funcionalidades implementadas

- **Geocodificación directa:** conversión de direcciones de texto a coordenadas geográficas (latitud/longitud) para ubicar eventos.
- **Geocodificación inversa:** obtención de direcciones legibles a partir de coordenadas seleccionadas en el mapa.
- **Búsqueda de lugares:** autocompletado y búsqueda de puntos de interés durante la creación de eventos.
- **Visualización:** renderizado de mapas en los detalles de los eventos y selectores de ubicación.

8.1.2. Componentes técnicos

- **Backend (integration_service):**
 - **Servicio principal:** el OpenStreetMapServiceV2 centraliza las llamadas a la API de Nominatim (<https://nominatim.openstreetmap.org>).
 - **Sistema de caché:** para cumplir con las políticas de uso de OSM (límite de 1 req/seg) y mejorar el rendimiento, se utiliza un sistema de caché en MongoDB gestionado por GeocodeCacheRepository.
 - **Identificación:** todas las peticiones incluyen un User-Agent específico (BasmatiCalendarApp/1.0) tal como requiere la política de uso del servicio.
- **Frontend:**
 - **Visualización de mapas:** en la página de detalles del evento (Event_Detail_Page.tsx), se utiliza un iframe de exportación de OSM para mostrar la ubicación exacta mediante una caja delimitadora (*bounding box*).
 - **Gestión de ubicaciones:** los componentes de creación y edición de eventos (Create_Event_Page.tsx) consumen el `http_integration_repository.ts` para interactuar con el backend y obtener datos geográficos en tiempo real.

8.1.3. Consideraciones de integración

- **Atribución:** se mantiene la atribución obligatoria a los colaboradores de OpenStreetMap en la interfaz de usuario.
- **Abstracción:** el sistema está preparado para soportar múltiples proveedores. En el esquema de eventos (event.py), el campo `map_provider` permite alternar entre OpenStreetMap y otros proveedores como `google_maps` si fuera necesario.

8.2. AWS S3 (imágenes)

El proyecto utiliza Amazon Simple Storage Service (AWS S3) como solución de almacenamiento de objetos para gestionar todos los recursos multimedia, principalmente imágenes de eventos y perfiles de usuario.

8.2.1. Funcionalidades implementadas

- **Almacenamiento persistente:** guardado de imágenes en buckets de AWS con alta disponibilidad.

- **Optimización automática:** compresión y redimensionamiento de imágenes en el servidor antes de la subida para minimizar el consumo de ancho de banda y almacenamiento.
- **URLs presigned:** generación de URLs temporales seguras para la descarga y visualización de recursos privados.
- **Gestión de ciclo de vida:** eliminación individual y en lote (*bulk delete*) de recursos huérfanos o eliminados por el usuario.

8.2.2. Componentes técnicos

- **Backend (integration_service):**
 - **Servicio principal:** el S3ImageService utiliza la librería boto3 para interactuar con la API de AWS.
 - **Procesamiento de imágenes:** implementa un método privado `_compress_image` que utiliza la librería Pillow (PIL) para:
 - Convertir imágenes a formato **WebP** (30% más eficiente que JPEG).
 - Redimensionar imágenes que superen los 1920x1080px.
 - Corregir la orientación EXIF (común en fotos de dispositivos móviles).
 - **Seguridad:** las credenciales (Access Key, Secret Key) se gestionan mediante variables de entorno y no se exponen al cliente.
- **Frontend:**
 - **Componente de subida:** el componente `Image_Uploader.tsx` proporciona una interfaz de arrastrar y soltar (*drag & drop*) para la carga de archivos.
 - **Arquitectura Clean:** la subida sigue el patrón de casos de uso mediante `Upload_Image_Use_Case.ts`, desacoplando la lógica de negocio de la implementación técnica del repositorio `Http_S3_Repository.ts`.

8.2.3. Configuración y límites:

- **Formatos permitidos:** JPEG, PNG, GIF, WebP, SVG y BMP.
- **Tamaño máximo:** configurado por defecto en 5MB para archivos sin procesar.
- **Expiración de URLs:** las URLs de visualización tienen una validez por defecto de 3600 segundos (1 hora).

8.3. SendGrid (email)

El proyecto utiliza SendGrid como plataforma de entrega de correo electrónico transaccional. Este servicio es fundamental para la comunicación con el usuario, permitiendo el envío de notificaciones críticas y resúmenes de actividad.

8.3.1. Funcionalidades implementadas

- **Notificaciones de comentarios:** envío automático de correos cuando un usuario recibe un comentario en uno de sus eventos o calendarios.
- **Correos de sistema:** envío de mensajes de texto plano o HTML para comunicaciones generales del sistema.
- **Modo de simulación:** en entornos de desarrollo donde no se configura una API Key, el servicio simula el envío exitoso para no bloquear el flujo de la aplicación.

8.3.2. Componentes técnicos

- **Backend (integration_service):**
 - **Servicio principal:** el EmailServiceV2 gestiona la comunicación con la API v3 de SendGrid (<https://api.sendgrid.com/v3/mail/send>).
 - **Construcción de plantillas:** el servicio incluye lógica interna para generar contenido HTML dinámico (como en `_build_digest_html`), asegurando que los correos mantengan la identidad visual de Basmati.
 - **Configuración:** se apoya en las variables de entorno `SENDGRID_API_KEY` y `SENDER_EMAIL` definidas en el archivo `.env`.

- **Cliente asíncrono:** utiliza `httpx.AsyncClient` para realizar las llamadas a la API de forma no bloqueante, optimizando el rendimiento del microservicio.
- **Integración con otros servicios:**
 - El `notification_service` actúa como orquestador, solicitando al `integration_service` el envío de correos basándose en las preferencias de notificación de cada usuario.

8.3.3. Configuración de seguridad

- **Remitente verificado:** todos los correos se envían desde una dirección verificada (`SENDER_EMAIL`), lo que garantiza una alta tasa de entregabilidad y evita carpetas de spam.
- **Gestión de errores:** El sistema captura errores de autenticación o límites de cuota de la API de SendGrid, devolviendo respuestas estructuradas mediante el esquema `EmailResponse`.

8.4. Google OAuth (autenticación)

El proyecto implementa el flujo de autenticación de Google OAuth 2.0 para permitir a los usuarios registrarse e iniciar sesión de forma segura sin necesidad de gestionar contraseñas locales. Esta integración es la base del sistema de identidad de Basmati.

8.4.1. Funcionalidades implementadas

- **Single Sign-On (SSO):** inicio de sesión rápido utilizando cuentas de Google existentes.
- **Sincronización de perfil:** obtención automática del nombre, correo electrónico y foto de perfil del usuario desde su cuenta de Google.
- **Flujo de autorización seguro:** implementación del flujo de código de autorización (*Authorization Code Flow*) con intercambio de tokens en el servidor.
- **Gestión de sesiones:** generación de un token JWT propio de Basmati tras validar la identidad con Google.

8.4.2. Componentes técnicos

- **Backend (`auth_service`):**
 - **Módulo de integración:** el archivo `google_oauth.py` contiene la lógica para:
 - Generar la URL de redirección a Google con los *scopes* necesarios (`openid`, `email`, `profile`).
 - Intercambiar el código de autorización por tokens de acceso mediante `exchange_code_for_tokens`.
 - Recuperar la información del usuario desde el endpoint de Google `UserInfo`.
 - **Seguridad:** utiliza el parámetro `state` para prevenir ataques CSRF durante el flujo de redirección.
 - **Configuración:** depende de las variables `GOOGLE_CLIENT_ID` y `GOOGLE_CLIENT_SECRET` configuradas en el panel de desarrolladores de Google Cloud.
- **Frontend:**
 - **Página de callback:** la página `Auth_Callback_Page.tsx` actúa como el punto de aterrizaje tras la autorización en Google.
 - **Gestión de tokens:** el servicio `auth_service.ts` se encarga de enviar el código al backend, recibir el JWT de Basmati y almacenarlo de forma segura en el `localStorage`.
 - **Contexto de usuario:** una vez autenticado, el `UserContext.tsx` se actualiza globalmente para reflejar el estado del usuario en toda la aplicación.

8.5. TeamUp (importación de calendarios)

Basmati permite la importación de calendarios desde la plataforma TeamUp, facilitando la migración de horarios académicos (como los de la Universidad de Málaga) al ecosistema de la aplicación.

8.5.1. Funcionalidades implementadas

- **Sincronización de metadatos:** obtención automática del nombre, descripción y color configurados en el calendario original de TeamUp.
- **Importación de eventos:** carga masiva de todos los eventos existentes en el calendario de TeamUp hacia el `event_service` de Basmati.
- **Soporte para API Keys:** integración con la API oficial de TeamUp requiriendo el Teamup-Token para el acceso a calendarios privados o institucionales.

8.5.2. Componentes técnicos

- **Backend** (**integration_service**): el método `_create_basmati_calendar_from_teamup` en `import_service.py` realiza peticiones a `https://api.teamup.com/{calendar_key}` para extraer la configuración y los eventos.
- **Frontend:** la interfaz en `Import_Calendar_Page.tsx` proporciona un formulario específico para introducir la Calendar Key y el API Token de TeamUp.

8.6. Google Calendar (importación de calendarios)

Integración con la API de Google Calendar para permitir a los usuarios traer sus calendarios personales o de trabajo a Basmati, manteniendo la coherencia entre sus diferentes herramientas de planificación.

8.6.1. Funcionalidades implementadas

- **Acceso mediante OAuth:** utiliza el token de acceso obtenido durante el inicio de sesión (si se concedieron los permisos necesarios) para acceder a la lista de calendarios del usuario.
- **Mapeo de atributos:** conversión de los campos de Google (título, descripción, fechas de inicio/fin, ubicación) al esquema de datos interno de Basmati.

8.6.2. Componentes técnicos

- **Backend (integration_service):** el método `import_from_google_calendar` en `import_service.py` orquesta la llamada a los endpoints de Google (`https://www.googleapis.com/calendar/v3/calendars/{id}`) y la posterior creación de registros en el `calendar_service` y `event_service`.
- **Frontend:** utiliza el `Import_Google_Calendar_Use_Case.ts` para manejar la lógica de importación, enviando el `google_access_token` de forma segura al backend.

8.6.3. Consideraciones comunes

- **Persistencia:** una vez importados, los calendarios y eventos se vuelven entidades nativas de Basmati, permitiendo al usuario editarlos o compartirlos independientemente de la fuente original.
- **Límites de API:** el sistema gestiona los *timeouts* y errores de cuota de ambos proveedores para asegurar una experiencia de usuario fluida durante procesos de importación voluminosos.

8.7. AWS EC2 (despliegue)

Herramienta usada para el despliegue de nuestra aplicación, **Amazon Elastic Compute Cloud (EC2)**, un servicio de IaaS (Infraestructura como Servicio) que proporciona capacidad de cómputo segura, elástica y escalable en la nube. La elección de este servicio nos ha permitido eliminar la dependencia de hardware físico, facilitando el lanzamiento de un servidor virtual (instancia) optimizado para nuestras necesidades en cuestión de minutos.

Tras analizar los requisitos de nuestra aplicación, hemos dimensionado y desplegado una instancia con las siguientes especificaciones técnicas:

- **Procesamiento (2 vCPUs):** contamos con dos procesadores virtuales que proporcionan la potencia de cómputo necesaria para gestionar las peticiones de los usuarios de forma simultánea, garantizando la fluidez del servicio y evitando latencias bajo carga de trabajo.
- **Almacenamiento (18 GB SSD - Amazon EBS):** hemos configurado un volumen de almacenamiento de estado sólido (SSD) de 18 GB mediante Elastic Block Store. Este espacio está destinado a albergar el código fuente de la aplicación, el sistema operativo y la gestión de logs, asegurando un rendimiento de lectura/escritura rápido y fiable.
- **Sistema operativo (Ubuntu 22.04 LTS):** hemos seleccionado la distribución Ubuntu 22.04 LTS por ser el estándar de la industria en entornos de servidor. Su arquitectura es altamente estable y segura, y al contar con soporte a largo plazo (LTS), garantiza parches de seguridad durante años. Además, el acceso mediante SSH nos otorga un control total sobre la consola para la gestión y personalización del entorno.

9. Nueva funcionalidad

Considerando todo aquello nuevo **aparte de la nueva funcionalidad de la API REST** (autenticación e importación funcional), las principales mejoras son las siguientes:

9.1. Mejora UI/UX de la pestaña de importación

La nueva interfaz de importación presenta una jerarquía visual optimizada bajo un estilo neobrutalista, facilitando la navegación entre proveedores (Google Calendar y Teamup) mediante un sistema de pestañas con transiciones suaves. Para simplificar la experiencia, las **opciones avanzadas de rango de fechas se han agrupado en una sección colapsable**, permitiendo una personalización precisa de hasta un año sin saturar la vista principal, todo respaldado por un sistema de ayuda contextual y validación de campos en tiempo real.

Se ha reforzado la transparencia técnica mediante indicadores de estado de conexión y badges que confirman la vinculación de cuentas. Durante el proceso, el sistema gestiona dinámicamente los estados de carga y errores para evitar duplicidades, culminando en un modal de resultados detallado que desglosa el éxito de la operación. Este resumen final proporciona un conteo exacto de eventos procesados y errores específicos, permitiendo una gestión clara y eficiente de cada tarea de importación.